

Programmazione e calcolo scientifico

Esercitazione 3 - Classi ed overload degli operatori

24 marzo 2026

1 Overload degli operatori in C++

Supponiamo di voler implementare in C++ una classe che modella i vettori di $\mathbb{R}^2$ e le corrispondenti operazioni di somma. Una possibilità per esempio sarebbe la seguente:

```
1  template<typename T>
2  struct vec2 {
3      T x_;
4      T y_;
5
6      /* Costruttore di default */
7      vec2()
8          : x_(T{0.0}), y_(T{0.0})
9      {}
10
11     /* Costruttore user-defined */
12     vec2(const T& px, const T& py)
13         : x_(px), y_(py)
14     {}
15
16     /* incremento */
17     vec2& incr(const vec2& other) {
18         x_ += other.x_;
19         y_ += other.y_;
20         return *this;
21     }
22
23     /* somma */
24     vec2 add(const vec2& other) const {
25         vec2 ret = *this;
26         ret.incr(other);
27         return ret;
28     }
```

```
29 };
```

Con tale implementazione sarebbe quindi possibile sommare due vettori nel modo seguente:

```
1 int main(void)
2 {
3     vec2<double> a(1.0, 2.0);
4     vec2<double> b(2.0, 3.0);
5
6     vec2<double> c = a.add(b);
7
8     std::cout << c.x_ << " " << c.y_ << "\n";
9     return 0;
10 }
```

La sintassi `vec2<double> c = a.add(b);` è però alquanto scomoda e si preferirebbe poter scrivere `vec2<double> c = a + b;`.

L'overload degli operatori è una funzionalità del C++ che permette di dare significato ad alcuni operatori (per esempio agli operatori aritmetici `+`, `-`, `*` e `/`) quando non utilizzati sui tipi di base (quali, per esempio, `int` o `double`).

L'overload degli operatori si effettua implementando specifici metodi all'interno delle classi sulle quali si vuole che l'operatore funzioni, per esempio:

```
1 template<typename T>
2 class vec2 {
3     T x_;
4     T y_;
5
6 public:
7     /* Costruttore di default */
8     vec2()
9         : x_(T{0.0}), y_(T{0.0})
10    {}
11
12    /* Costruttore user-defined */
13    vec2(const T& px, const T& py)
14        : x_(px), y_(py)
15    {}
16
17    /* Restituiscono i valori di x e y */
18    T x() const { return x_; }
19    T y() const { return y_; }
20
21    /* Implementazione canonica dell'incremento */
```

```

22  vec2& operator+=(const vec2& other) {
23      x_ += other.x_;
24      y_ += other.y_;
25      return *this;
26  }
27
28  /* Implementazione canonica della somma */
29  vec2 operator+(const vec2& other) const {
30      vec2 ret = *this;
31      ret += other;
32      return ret;
33  }
34
35
36  /* Implementazione canonica di *= con uno scalare */
37  vec2& operator*=(const T& s) {
38      x_ *= s;
39      y_ *= s;
40      return *this;
41  }
42
43  /* Implementazione canonica del prodotto vettore per scalare */
44  vec2 operator*(const T& s) const {
45      vec2 ret = *this;
46      ret *= s;
47      return ret;
48  }
49  };

```

Il codice più sopra implementa lo scheletro di una classe `vec2` che rappresenta un vettore di $\mathbb{R}^2$. Si ricordi che le definizioni di classi template vanno inserite all'interno di file header con estensione `.hpp`. La classe può essere usata come segue:

```

1  int main(void)
2  {
3      vec2<double> a(1.0, 2.0);
4      vec2<double> b(2.0, 3.0);
5
6      vec2<double> c = a+b;
7
8      std::cout << c.x() << " " << c.y() << "\n";
9      return 0;
10 }

```

La stampa del vettore effettuata come nel frammento di codice più sopra risulta scomoda. Per questo motivo possiamo anche definire un overload di `operator<<`, in modo da specificare cosa dev'essere fatto quando un vettore viene inserito in uno stream. In questo caso l'overload si definisce come funzione libera e non come membro della classe `vec2`. Nel seguente frammento di codice si può osservare come si implementa in modo canonico l'overload di `operator<<` per la nostra classe `vec2`.

```
1 template<typename T>
2 std::ostream&
3 operator<<(std::ostream& os, const vec2<T>& v)
4 {
5     os << "(" << v.x() << "," << v.y() << ")";
6     return os;
7 }
```

Notare che la funzione `operator<<` ha tipo

$$\text{operator<<} : \text{std::ostream\&} \times \text{vec2<T>} \rightarrow \text{std::ostream\&}$$

Se abbreviamo `std::ostream&` con $\mathbb{S}$ e `operator<<` con solo `<<`, otteniamo qualcosa di molto simile a quanto visto nella lezione in aula sugli streams:

$$<< : \mathbb{S} \times \text{vec2<T>} \rightarrow \mathbb{S}$$

Ecco che diventa quindi possibile stampare il vettore semplicemente con

```
1 int main(void)
2 {
3     vec2<double> a(1.0, 2.0);
4     vec2<double> b(2.0, 3.0);
5
6     vec2<double> c = a+b;
7
8     std::cout << c << "\n";
9     return 0;
10 }
```

Non ci si faccia intimorire dalla sintassi per l'implementazione degli operatori: sono semplici metodi ma con un nome particolare. A dimostrazione di ciò si osservi il codice seguente, che chiama `operator+` come se fosse un normale metodo:

```
1 int main(void)
2 {
3     vec2<double> a(1.0, 2.0);
4     vec2<double> b(2.0, 3.0);
5 }
```

```

6   vec2<double> c = a.operator+(b);
7
8   std::cout << c << "\n";
9   return 0;
10  }

```

Le implementazioni date sopra per gli operatori sono quelle *canoniche*; si faccia attenzione ai parametri degli operatori e ai loro tipi di ritorno. Per esempio, considerando gli operatori legati alla somma:

- `operator+=` prende un const reference e restituisce un reference
- `operator+` prende un const reference, restituisce per valore ed è dichiarato const
- Attenzione che `operator+=` restituisce `*this`, cioè un riferimento all'oggetto stesso su cui è chiamato
- Attenzione che `operator+` restituisce un nuovo oggetto

Per il momento si prenda per buono che gli operatori devono rispettare quanto elencato sopra. Si prenda visione della pagina

<https://en.cppreference.com/w/cpp/language/operators.html>

2 Obiettivo dell'esercitazione

L'obiettivo di questa esercitazione è l'implementazione di una classe `rational` che implementi i numeri razionali (ovvero quelli appartenenti all'insieme $\mathbb{Q}$). Lo scheletro iniziale di tale classe potrebbe essere il seguente:

```

1  template<typename I> requires std::integral<I>
2  class rational
3  {
4      I num_;
5      I den_;
6  };

```

Si ricordi che `template<typename I>` può approssimativamente essere letto come “ $\forall I$ ” mentre `requires std::integral<I>` può approssimativamente essere letto come “tale che I è un intero”. Si ricordi inoltre di includere l'header `<concepts>` per `std::integral`. Si implementi quanto segue:

- Un costruttore di default che inizializzi opportunamente `num_` e `den_`.
- Un costruttore user-defined che inizializzi `num_` e `den_` con i valori specificati dall'utente. Se viene specificato il valore zero per `den_` il programma NON deve uscire con errore ma deve gestire opportunamente questo caso e continuare a funzionare. Si prenda per esempio spunto da Inf e NaN dei numeri floating point e si documenti opportunamente le proprie scelte: per

esempio `num` diverso da zero mentre `den` uguale a zero potrebbe rappresentare `Inf`, mentre `num = den = zero` potrebbe implementare il `NaN`. Di conseguenza, per esempio, un qualsiasi `rational` sommato ad `Inf` deve dare `Inf`, mentre un qualsiasi `rational` sommato a `NaN` deve dare `NaN`.

- I metodi `num()` e `den()` per ottenere rispettivamente numeratore e denominatore.
- L'operatore `<<` per la stampa di un oggetto della classe `rational`.
- Le operazioni `+=`, `+`, `-=`, `-`, `*=`, `*`, `/=`, `/`.

È inoltre richiesto che la rappresentazione interna delle frazioni (quindi `num_` e `den_`) sia sempre correttamente semplificata. Per esempio, il programma qui sotto non dovrà stampare `3/6` ma `1/2`.

```
1 int main(void)
2 {
3     rational<int> r(3,6);
4     std::cout << r << "\n";
5     return 0;
6 }
```

3 Modalità di svolgimento e consegna

Dev'essere consegnato quanto richiesto più sopra, includendo un `CMakeLists.txt` che automatizzi la compilazione dei programmi. I file devono essere posizionati in una cartella chiamata `esercitazione3` nel proprio repository.

La consegna delle esercitazioni è obbligatoria e da effettuarsi entro i termini prescritti. Il tempo assegnato per lo svolgimento di questa esercitazione è di una settimana. Termine ultimo per la consegna di questa esercitazione sono le ore 23.59 di Mercoledì 01/04/2025: fa fede il timestamp del commit taggato `consegna_es3` sul proprio repository. Eventuali giustificazioni vanno presentate a `matteo.cicuttin@polito.it` entro il termine ultimo previsto per la consegna dell'esercitazione.

Si ricordi che l'uso di strumenti di intelligenza artificiale è scoraggiato ed in ogni caso dev'essere dichiarato esplicitamente all'interno dei sorgenti consegnati.

All'esame potrebbe essere richiesto di mostrare il funzionamento del codice, pertanto ci si assicuri che il codice nel proprio repository sia funzionante.